# Command Injection

Command injection is the abuse of an application's behaviour to execute commands on the operating system, using the same privileges that the application on a device is running with.

example: achieving command injection on a web server running as a user named `joe` will execute commands under this `joe` user - and therefore obtain any permissions that `joe` has.

Command injection is also often known as “Remote Code Execution” (RCE) because of the ability to remotely execute code within an application. These vulnerabilities are often the most lucrative to an attacker because it means that the attacker can directly interact with the vulnerable system.

For example, an attacker may read system or user files, data, and things of that nature.

Command injection was one of the top ten vulnerabilities reported by Contrast Security’s AppSec intelligence report in 2019. (Contrast Security AppSec., 2019(opens in new tab)). Moreover, the OWASP framework constantly proposes vulnerabilities of this nature as one of the top ten vulnerabilities of a web application (OWASP framework(opens in new tab)).

## Discovering Command Injection

In this code snippet, the application takes data that a user enters in an input field named `$title` to search a directory for a song title. Let’s break this down into a few simple steps.

```php
<?php
$songs = "/var/www/html/songs"
if (isset $_GET["title"])) {
    $title = $_GET["title"];
    $command = "grep $title /var/www/html/songtitle.txt";
    $search = exec($command);
    if ($search == "") {
        $return = "<p>The requested song</p><p> $title does </p><b>not</b><p> exist!</p>";
    } else {
        $return = "<p>The requested song</p><p> $title does </p><b>exist!</b>";
      }
     echo $return;
}
?>
```

1.
2.
3.
4.

**1.** The application stores MP3 files in a directory contained on the operating system.

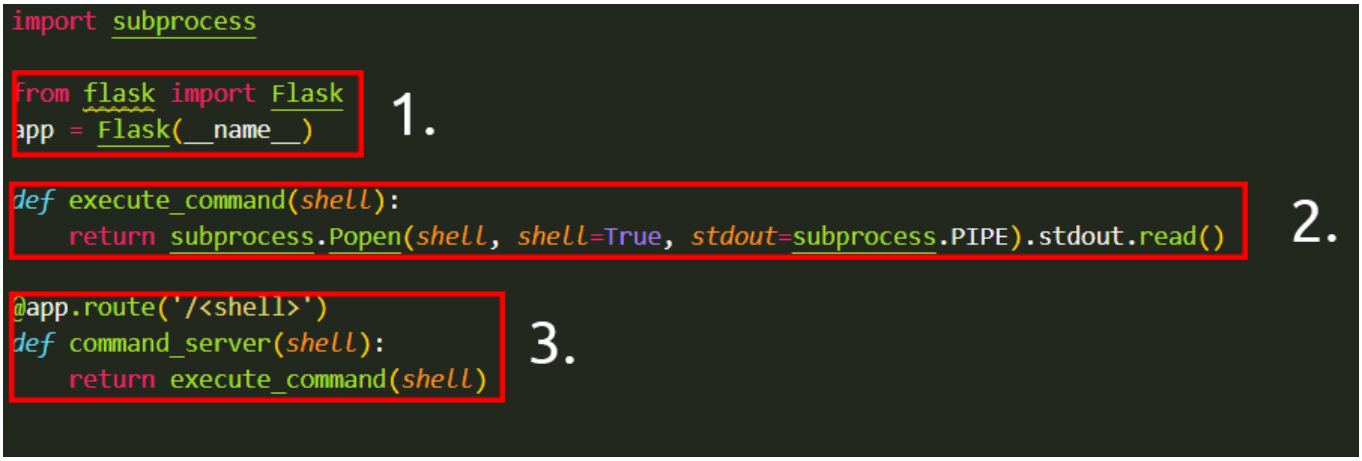
**2.** The user inputs the song title they wish to search for. The application stores this input into the `$title` variable.

**3.** The data within this `$title` variable is passed to the command `grep` to search a text file named *songtitle.__txt* for the entry of whatever the user wishes to search for.

**4.** The output of this search of *songtitle.__txt* will determine whether the application informs the user that the song exists or not.

An attacker could abuse this application by injecting their own commands for the application to execute. Rather than using `grep` to search for an entry in `songtitle.txt`, they could ask the application to read data from a more sensitive file.

Abusing applications in this way can be possible no matter the programming language the application uses. As long as the application processes and executes it, it can result in command injection. For example, this code snippet below is an application written in Python.




1. The "flask" package is used to set up a web server
2. A function that uses the "subprocess" package to execute a command on the device
3. We use a route in the webserver that will execute whatever is provided. For example, to execute `whoami`, we'd need to visit http://flaskapp.thm/whoami

## Exploiting Command Injection

Command Injection can be detected in mostly one of two ways:

1. Blind command injection
2. Verbose command injection

| Method | Description |
| --- | --- |
| Blind | This type of injection is where there is no direct output from the application when testing payloads. You will have to investigate the behaviours of the application to determine whether or not your payload was successful. |
| Verbose | This type of injection is where there is direct feedback from the application once you have tested a payload. For example, running the `whoami` command to see what user the application is running under. The web application will output the username on the page directly. |

## Detecting Blind Command Injection

For this type of command injection, we will need to use payloads that will cause some time delay. For example, the `ping` and `sleep` commands are significant payloads to test with. Using `ping` as an example, the application will hang for *x* seconds in relation to how many *pings* you have specified.

Another method of detecting blind command injection is by forcing some output. This can be done by using redirection operators such as `>`. For example, we can tell the web application to execute commands such as `whoami` and redirect that to a file. We can then use a command such as `cat` to read this newly created file's contents.

The `curl` command is a great way to test for command injection. This is because you are able to use `curl` to deliver data to and from an application in your payload. Take this code snippet below as an example, a simple curl payload to an application is possible for command injection.

```
curl http://vulnerable.app/process.php%3Fsearch%3DThe%20Beatles%3B%20whoami
```

## Detecting Verbose Command Injection

For example, the output of commands such as `ping` or `whoami` is directly displayed on the web application.

## Useful Payloads

Linux:

| Payload | Description |
| --- | --- |
| whoami | See what user the application is running under. |
| ls | List the contents of the current directory. You may be able to find files such as configuration files, environment files (tokens and application keys), and many more valuable things. |
| ping | This command will invoke the application to hang. This will be useful in testing an application for blind command injection. |
| sleep | This is another useful payload in testing an application for blind command injection, where the machine does not have `ping` installed. |
| nc | Netcat can be used to spawn a reverse shell onto the vulnerable application. You can use this foothold to navigate around the target machine for other services, files, or potential means of escalating privileges. |

Windows:

| Payload | Description |
| --- | --- |
| whoami | See what user the application is running under. |
| dir | List the contents of the current directory. You may be able to find files such as configuration files, environment files (tokens and application keys), and many more valuable things. |
| ping | This command will invoke the application to hang. This will be useful in testing an application for blind command injection. |
| timeout | This command will also invoke the application to hang. It is also useful for testing an application for blind command injection if the `ping` command is not installed. |

## Remediating Command Injection

minimal use of potentially dangerous functions or libraries in a programming language to filtering input without relying on a user's input

**Vulnerable Functions**

In PHP, many functions interact with the operating system to execute commands via shell; these include:

- Exec
- Passthru
- System

Here, the application will only accept and process numbers that are inputted into the form. This means that any commands such as `whoami` will not be processed.

```
<input type="text" id="ping" name="ping" pattern="[0-9]+"></input>  1.
<?php
echo passthru("/bin/ping -c 4 "$_GET["ping"].'); 2.

?>
```

1. The application will only accept a specific pattern of characters (the digits  0-9)
2. The application will then only proceed to execute this data which is all numerical

**Input sanitisation**
input field that only accepts numerical data or removes any special characters such as `>` , `&` and `/`

In the snippet below, the `filter_input` PHP function(opens in new tab) is used to check whether or not any data submitted via an input form is a number or not

```php
<?php

if (!filter_input(INPUT_GET, "number", FILTER_VALIDATE_NUMBER)) {

}
```

Bypassing Filters

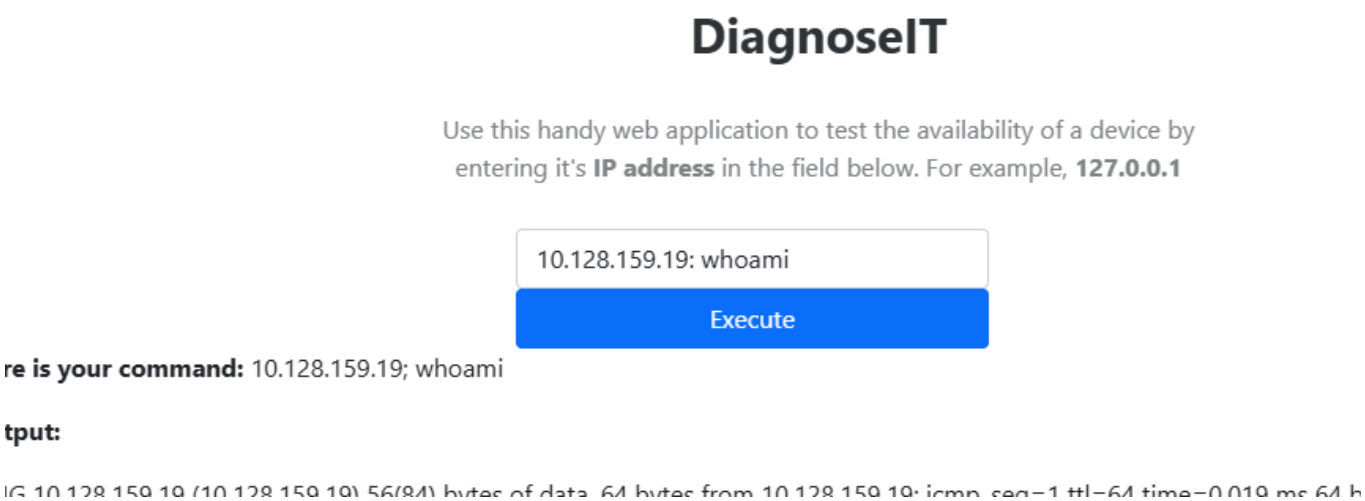
Applications will employ numerous techniques in filtering and sanitising data that is taken from a  user's input. These filters will restrict you to specific payloads; however, we can abuse the logic behind an application to bypass these filters

an application may strip out quotation marks; we can instead use the hexadecimal value of this to achieve the same result.

```
$payload = "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"
```

## Practical: Command Injection

here I had to type the IP in as its a valid input then use the semi-colon which allows us to inject other commands




I tried another command to get the flag in the directory home/tryhackme/flag.txt
as seen below
using this command: 10.128.159.19; cat /home/tryhackme/flag.txt